

01 — O Pipeline de Execução da CPU

Pré-requisito: Phase 0 completa (em particular, saber ler assembly em nível básico ajuda, mas não é exigido).

1. Intuição: uma instrução não acontece "de uma vez"

Quando você pensa em "a CPU executa uma instrução", a imagem mental costuma ser instantânea — a instrução chega, acontece, pronto. Isso é falso, e entender **por que** é falso é a base de tudo que vem depois neste módulo.

Uma instrução de máquina (tipo `ADD R1, R2, R3` — "some R2 com R3, guarde em R1") passa por **etapas físicas distintas** dentro do chip, cada uma feita por um circuito diferente, uma depois da outra. Isso não é escolha de design arbitrária — é consequência de como circuitos elétricos funcionam: cada etapa (buscar a instrução, entender o que ela pede, executar a operação, guardar o resultado) precisa de um circuito fisicamente diferente, e um circuito não pode fazer a etapa seguinte antes de a anterior terminar de "assentar" eletricamente.

2. As quatro etapas clássicas

FETCH	busca a instrução da memória (endereço vem do program counter)
DECODE	descobre o que a instrução pede: qual operação, quais registradores
EXECUTE	a ULA (unidade lógica-aritmética) faz o cálculo de verdade
WRITEBACK	o resultado é escrito de volta no registrador de destino

Cada etapa é um circuito separado dentro do chip. Sem pipelining, a CPU faria: fetch da instrução 1, decode da instrução 1, execute da instrução 1, writeback da instrução 1, **só então** fetch da instrução 2. Cada instrução ocupa a CPU inteira, do início ao fim, sozinha.

3. Por que isso é um desperdício monstruoso

Enquanto a etapa EXECUTE da instrução 1 está rodando, o circuito de FETCH está **parado, sem fazer nada**. Ele já terminou seu trabalho da instrução 1 e só vai ser usado de novo quando a instrução 1 terminar inteira. Isso é fisicamente um circuito caríssimo (fabricado em silício, ocupando espaço no chip, consumindo energia) ficando **ocioso** na maior parte do tempo.

4. Pipelining: sobrepor as etapas

A solução é óbvia uma vez que você vê o desperdício: **por que não deixar o circuito de FETCH começar a buscar a instrução 2 assim que ele termina de buscar a instrução 1**, sem esperar a instrução 1 terminar as outras três etapas?

Sem pipeline (uma instrução por vez, do início ao fim):

Ciclo:	1	2	3	4	5	6	7	8
Instr 1:	FETCH	DECODE	EXEC	WB				
Instr 2:					FETCH	DECODE	EXEC	WB

(8 ciclos para 2 instruções)

Com pipeline (etapas sobrepostas):

Ciclo:	1	2	3	4	5	
Instr 1:	FETCH	DECODE	EXEC	WB		
Instr 2:		FETCH	DECODE	EXEC	WB	
Instr 3:			FETCH	DECODE	EXEC	WB

(5 ciclos para 3 instruções!)

Cada circuito (FETCH, DECODE, EXECUTE, WRITEBACK) está **sempre ocupado**, cada um trabalhando numa instrução diferente, na mesma "hora do relógio" (ciclo). É a mesma ideia de uma linha de montagem de fábrica: enquanto a estação 4 finaliza o produto A, a estação 3 já está montando o produto B, a estação 2 já está preparando o produto C.

PERGUNTA RÁPIDA 1: numa linha de montagem com 4 estações, depois que a esteira está "cheia" (regime estável), quantos produtos saem prontos a cada "tick" da esteira? E quantos "ticks" leva pra montar UM produto do zero, do início ao fim?

► Resposta

Um produto sai pronto a cada tick (throughput = 1/ciclo, no regime estável). Mas um produto específico ainda leva 4 ticks pra passar pelas 4 estações (latência = 4 ciclos). Essa distinção — quantos saem por tick vs quanto tempo um leva do início ao fim — é exatamente o assunto do arquivo 05 (latência vs bandwidth). Guarde essa intuição.

5. O nome técnico: throughput vs latência de uma instrução

- **Latência de uma instrução:** quantos ciclos ela leva, do fetch ao writeback, sozinha. No exemplo acima, 4 ciclos (uma etapa por ciclo).
- **Throughput do pipeline:** quantas instruções **terminam** por ciclo, em regime contínuo. No exemplo acima, 1 instrução terminada por ciclo, depois que o pipeline "encheu".

Pipelining **não** deixa uma instrução individual mais rápida — os 4 ciclos de latência continuam os mesmos. O que melhora é o **throughput**: quantas instruções terminam por segundo, quando você tem muitas pra rodar em sequência (que é sempre o caso — um programa real tem milhões de instruções).

6. O problema real: pipeline stall (bolha)

Pipelining assume que a instrução seguinte pode começar sem esperar a anterior terminar. Isso quebra quando existe **dependência** entre instruções:

```
ADD R1, R2, R3    ; R1 = R2 + R3
SUB R4, R1, R5    ; R4 = R1 - R5    <- precisa do R1 que a instrução anterior
calculou!
```

SUB não pode fazer seu **EXECUTE** até **ADD** ter feito seu **WRITEBACK** (senão pegaria um valor velho de **R1**). O pipeline precisa **parar** (inserir uma "bolha") até o valor estar pronto:

Ciclo:	1	2	3	4	5	6	
ADD:	FETCH	DECODE	EXEC	WB			
SUB:		FETCH	DECODE	[BOLHA]	EXEC	WB	<- esperando o R1 ficar pronto

Esse tipo de dependência (chamada *data hazard*) é uma das razões pelas quais "mais instruções" nem sempre significa "mais rápido linearmente" — o pipeline pode ficar parcialmente parado.

PERGUNTA RÁPIDA 2: por que reescrever um loop pra que iterações consecutivas **não** dependam do resultado uma da outra (ex: somar dois arrays independentes em vez de uma soma acumulada sequencial) tende a rodar mais rápido numa CPU pipelined, mesmo fazendo o "mesmo tanto" de trabalho aritmético?

► Resposta

Porque instruções independentes não criam data hazards entre si — o pipeline consegue manter todas as etapas ocupadas sem inserir bolhas, aproximando o throughput real do throughput teórico (1 instrução terminando por ciclo). Uma soma acumulada sequencial (`soma += arr[i]`, onde cada iteração depende do resultado da anterior) força uma cadeia de dependências que impede sobreposição — cada instrução de soma precisa esperar a anterior terminar. Essa é a raiz de por que reduções bem escritas usam múltiplos acumuladores independentes (você vai reencontrar essa ideia no Módulo 07 do roadmap, sobre kernels de redução em GPU).

7. Por que isso importa para CUDA

GPUs não usam pipelining de instrução exatamente do mesmo jeito que CPUs (elas escondem latência trocando de *warp* em vez de sobrepor etapas de uma única instrução — assunto da Phase 2), mas o **vocabulário e o raciocínio são os mesmos**: distinguir latência de throughput, reconhecer que dependência entre operações trava paralelismo, entender que "tempo de uma operação sozinha" e "quantas operações terminam por segundo em massa" são perguntas diferentes. Cada uma dessas ideias reaparece, com nome novo, quando você estudar como um Streaming Multiprocessor esconde a latência de acesso à memória global trocando de warp.

8. Erros comuns

- Achar que pipelining faz cada instrução individual rodar mais rápido. Não faz — melhora quantas terminam por unidade de tempo, não quanto tempo uma leva sozinha.
- Ignorar dependência de dado como causa de lentidão — "eu fiz o mesmo número de operações, por que ficou mais lento?" quase sempre tem uma cadeia de dependência como resposta.
- Confundir "a CPU tem 4 núcleos" com pipelining — são conceitos diferentes. Pipelining acontece **dentro** de um único núcleo, sobrepondo etapas de instruções sequenciais. Múltiplos núcleos é

paralelismo de outro tipo (você roda programas/threads diferentes simultaneamente, cada núcleo com seu próprio pipeline).

PREDICT BEFORE RUNNING (conceitual, sem código)

Considere um pipeline de 5 estágios (fetch, decode, execute, memory-access, writeback), rodando 100 instruções **sem nenhuma dependência entre elas**.

1. Quantos ciclos, aproximadamente, o programa inteiro leva? (dica: quantos ciclos pra "encher" o pipeline pela primeira vez, mais quantos ciclos depois disso.)
2. Se metade das instruções tivesse uma dependência forçando 2 ciclos de bolha cada, o tempo total dobraria, mais que dobraria, ou menos que dobraria? Por quê?

► Explicação

1. As primeiras 5 instruções levam 5 ciclos pra "encher" o pipeline completamente (a primeira instrução sai pronta no ciclo 5). Depois disso, uma instrução termina por ciclo. Total: 5 (para encher) + 99 (as restantes, uma por ciclo) = 104 ciclos, não 500 — a sobreposição é o que faz a diferença.
2. Menos que dobraria. Cada bolha de 2 ciclos adiciona 2 ciclos de atraso **localizado**, mas o pipeline continua funcionando para as instruções não afetadas — não é como se o programa inteiro rodasse "do zero, sem pipeline". Metade das instruções (50) causando 2 ciclos de bolha cada adiciona aproximadamente 100 ciclos extras aos 104 originais — quase dobra, mas o raciocínio importa mais que o número exato aqui: bolhas custam **linear** ao número de dependências, não multiplicam o tempo total.